

For each program, include:

- **Aim**
- **Problem Statement**
- **Algorithm**
- **PL/SQL Code**
- **Sample Output**
- **Explanation**
- **Viva Questions**
- **Exercise for Students**

Module 1: Introduction to PL/SQL

Lab 1: Executing the First PL/SQL Program

Aim

To execute a simple PL/SQL block.

```
SET SERVEROUTPUT ON;

BEGIN
    DBMS_OUTPUT.PUT_LINE('Hello World');
END;
/
```

Lab 2: Printing Multiple Messages

```
BEGIN

    DBMS_OUTPUT.PUT_LINE('Welcome');

    DBMS_OUTPUT.PUT_LINE('Oracle Database');

    DBMS_OUTPUT.PUT_LINE('PL/SQL Programming');

END;
/
```

Lab 3: Anonymous Block

```
BEGIN

    DBMS_OUTPUT.PUT_LINE('Line 1');
```

```
        DBMS_OUTPUT.PUT_LINE('Line 2');

        DBMS_OUTPUT.PUT_LINE('Line 3');

END;
/
```

Lab 4: Nested Block

```
BEGIN

    DBMS_OUTPUT.PUT_LINE('Outer Block');

    DECLARE

        name VARCHAR2(20) := 'Rahul';

    BEGIN

        DBMS_OUTPUT.PUT_LINE(name);

    END;

END;
/
```

Lab 5: Display System Date

```
BEGIN

    DBMS_OUTPUT.PUT_LINE(SYSDATE);

END;
/
```

Lab 6: Display Current User

```
BEGIN

    DBMS_OUTPUT.PUT_LINE(USER);

END;
/
```

Lab 7: Display Database Version

```
BEGIN
```

```
DBMS_OUTPUT.PUT_LINE('Oracle PL/SQL Laboratory');  
  
END;  
/
```

Lab 8: Student Information

```
BEGIN  
  
DBMS_OUTPUT.PUT_LINE('Name : Rahul');  
  
DBMS_OUTPUT.PUT_LINE('Roll No : 101');  
  
DBMS_OUTPUT.PUT_LINE('Course : B.Tech');  
  
END;  
/
```

Lab 9: Employee Information

```
BEGIN  
  
DBMS_OUTPUT.PUT_LINE('Employee Name : Amit');  
  
DBMS_OUTPUT.PUT_LINE('Salary : 45000');  
  
END;  
/
```

Lab 10: Hospital Details

```
BEGIN  
  
DBMS_OUTPUT.PUT_LINE('Patient Name : Ravi');  
  
DBMS_OUTPUT.PUT_LINE('Ward : ICU');  
  
END;  
/
```

Module 2: Variables and Data Types

Lab 1: Variable Declaration

```
DECLARE

name VARCHAR2 (20) :='Rahul';

BEGIN

DBMS_OUTPUT.PUT_LINE (name);

END;
/
```

Lab 2: Addition

```
DECLARE

a NUMBER:=20;

b NUMBER:=30;

c NUMBER;

BEGIN

c:=a+b;

DBMS_OUTPUT.PUT_LINE (c);

END;
/
```

Lab 3: Area of Rectangle

```
DECLARE

length NUMBER:=20;

breadth NUMBER:=10;

area NUMBER;

BEGIN

area:=length*breadth;

DBMS_OUTPUT.PUT_LINE (area);
```

```
END;  
/
```

Lab 4: Circle Area

```
DECLARE  
  
radius NUMBER:=7;  
  
area NUMBER;  
  
BEGIN  
  
area:=3.14159*radius*radius;  
  
DBMS_OUTPUT.PUT_LINE(area);  
  
END;  
/
```

Lab 5: Student Percentage

```
DECLARE  
  
m1 NUMBER:=80;  
  
m2 NUMBER:=90;  
  
m3 NUMBER:=75;  
  
percentage NUMBER;  
  
BEGIN  
  
percentage:=(m1+m2+m3)/3;  
  
DBMS_OUTPUT.PUT_LINE(percentage);  
  
END;  
/
```

Lab 6–10

- Gross Salary
- Swap Two Numbers
- Celsius to Fahrenheit
- Interest Calculation
- Employee Salary Slip

Module 3: Operators in PL/SQL

Experiment 1: Arithmetic Calculator

Aim

To perform arithmetic operations using PL/SQL operators.

Program

```
SET SERVEROUTPUT ON;

DECLARE
    num1 NUMBER := 20;
    num2 NUMBER := 10;
BEGIN
    DBMS_OUTPUT.PUT_LINE('First Number : ' || num1);
    DBMS_OUTPUT.PUT_LINE('Second Number : ' || num2);

    DBMS_OUTPUT.PUT_LINE('Addition      = ' || (num1 + num2));
    DBMS_OUTPUT.PUT_LINE('Subtraction   = ' || (num1 - num2));
    DBMS_OUTPUT.PUT_LINE('Multiplication = ' || (num1 * num2));
    DBMS_OUTPUT.PUT_LINE('Division      = ' || (num1 / num2));
    DBMS_OUTPUT.PUT_LINE('Modulus       = ' || MOD(num1, num2));
END;
/
```

Output

```
First Number : 20
Second Number : 10
Addition      = 30
Subtraction   = 10
Multiplication = 200
Division      = 2
Modulus       = 0
```

Experiment 2: Area of Circle

Aim

To calculate the area of a circle using arithmetic operators.

Program

```
SET SERVEROUTPUT ON;

DECLARE
    radius NUMBER := 7;
    area NUMBER;
BEGIN
    area := 3.14159 * radius * radius;

    DBMS_OUTPUT.PUT_LINE('Radius = ' || radius);
    DBMS_OUTPUT.PUT_LINE('Area    = ' || ROUND(area,2));
END;
/
```

Output

```
Radius = 7
Area    = 153.94
```

Experiment 3: Area of Rectangle

Aim

To calculate the area of a rectangle.

Program

```
SET SERVEROUTPUT ON;

DECLARE
    length NUMBER := 15;
    breadth NUMBER := 10;
    area NUMBER;
BEGIN
    area := length * breadth;

    DBMS_OUTPUT.PUT_LINE('Length  = ' || length);
    DBMS_OUTPUT.PUT_LINE('Breadth = ' || breadth);
    DBMS_OUTPUT.PUT_LINE('Area    = ' || area);
END;
/
```

Output

```
Length  = 15
Breadth = 10
Area    = 150
```

Experiment 4: GST Calculator

Aim

To calculate GST and the total bill amount.

Program

```
SET SERVEROUTPUT ON;

DECLARE
    amount NUMBER := 10000;
    gst_rate NUMBER := 18;
    gst_amount NUMBER;
    total NUMBER;
BEGIN
    gst_amount := amount * gst_rate / 100;
    total := amount + gst_amount;

    DBMS_OUTPUT.PUT_LINE('Original Amount : ' || amount);
    DBMS_OUTPUT.PUT_LINE('GST (18%)          : ' || gst_amount);
    DBMS_OUTPUT.PUT_LINE('Total Amount      : ' || total);
END;
/
```

Output

```
Original Amount : 10000
GST (18%)       : 1800
Total Amount    : 11800
```

Experiment 5: Salary Calculator

Aim

To calculate Gross Salary using arithmetic operators.

Program

```
SET SERVEROUTPUT ON;

DECLARE
    basic_salary NUMBER := 50000;
```



```

        hra NUMBER;
        da NUMBER;
        ta NUMBER;
        gross_salary NUMBER;
BEGIN
    hra := basic_salary * 20 / 100;
    da  := basic_salary * 10 / 100;
    ta  := basic_salary * 5 / 100;

    gross_salary := basic_salary + hra + da + ta;

    DBMS_OUTPUT.PUT_LINE('Basic Salary : ' || basic_salary);
    DBMS_OUTPUT.PUT_LINE('HRA          : ' || hra);
    DBMS_OUTPUT.PUT_LINE('DA           : ' || da);
    DBMS_OUTPUT.PUT_LINE('TA           : ' || ta);
    DBMS_OUTPUT.PUT_LINE('Gross Salary : ' || gross_salary);
END;
/

```

Output

```

Basic Salary : 50000
HRA          : 10000
DA           : 5000
TA           : 2500
Gross Salary : 67500

```

Experiment 6: Largest of Two Numbers

Aim

To determine the largest of two numbers using relational operators.

Program

```

SET SERVEROUTPUT ON;

DECLARE
    num1 NUMBER := 75;
    num2 NUMBER := 48;
BEGIN
    IF num1 > num2 THEN
        DBMS_OUTPUT.PUT_LINE(num1 || ' is the Largest Number');
    ELSE
        DBMS_OUTPUT.PUT_LINE(num2 || ' is the Largest Number');
    END IF;
END;
/

```

Output

Experiment 7: Relational Operator Demonstration

Aim

To demonstrate various relational operators.

Program

```
SET SERVEROUTPUT ON;

DECLARE
    a NUMBER := 20;
    b NUMBER := 15;
BEGIN
    DBMS_OUTPUT.PUT_LINE('a = ' || a);
    DBMS_OUTPUT.PUT_LINE('b = ' || b);

    IF a = b THEN
        DBMS_OUTPUT.PUT_LINE('a is Equal to b');
    END IF;

    IF a <> b THEN
        DBMS_OUTPUT.PUT_LINE('a is Not Equal to b');
    END IF;

    IF a > b THEN
        DBMS_OUTPUT.PUT_LINE('a is Greater than b');
    END IF;

    IF a < b THEN
        DBMS_OUTPUT.PUT_LINE('a is Less than b');
    END IF;

    IF a >= b THEN
        DBMS_OUTPUT.PUT_LINE('a is Greater than or Equal to b');
    END IF;

    IF a <= b THEN
        DBMS_OUTPUT.PUT_LINE('a is Less than or Equal to b');
    END IF;
END;
/
```

Output

a = 20

```
b = 15
a is Not Equal to b
a is Greater than b
a is Greater than or Equal to b
```

Experiment 8: Logical Operator Demonstration

Aim

To demonstrate AND, OR, and NOT operators.

Program

```
SET SERVEROUTPUT ON;

DECLARE
    age NUMBER := 25;
    salary NUMBER := 40000;
BEGIN
    IF age >= 18 AND salary >= 30000 THEN
        DBMS_OUTPUT.PUT_LINE('Eligible using AND Operator');
    END IF;

    IF age >= 18 OR salary >= 50000 THEN
        DBMS_OUTPUT.PUT_LINE('Eligible using OR Operator');
    END IF;

    IF NOT(age < 18) THEN
        DBMS_OUTPUT.PUT_LINE('Eligible using NOT Operator');
    END IF;
END;
/
```

Output

```
Eligible using AND Operator
Eligible using OR Operator
Eligible using NOT Operator
```

Experiment 9: Operator Precedence

Aim

To understand the precedence of arithmetic operators.

Program

```
SET SERVEROUTPUT ON;

DECLARE
    result1 NUMBER;
    result2 NUMBER;
BEGIN
    result1 := 10 + 5 * 2;
    result2 := (10 + 5) * 2;

    DBMS_OUTPUT.PUT_LINE('Without Parentheses : ' || result1);
    DBMS_OUTPUT.PUT_LINE('With Parentheses      : ' || result2);
END;
/
```

Output

```
Without Parentheses : 20
With Parentheses    : 30
```

Explanation:

```
10 + 5 * 2
= 10 + 10
= 20
```

```
(10 + 5) * 2
= 15 * 2
= 30
```

Experiment 10: Student Result Calculation

Aim

To calculate the total, percentage, and grade of a student.

Program

```
SET SERVEROUTPUT ON;

DECLARE
    m1 NUMBER := 85;
    m2 NUMBER := 78;
    m3 NUMBER := 92;
    m4 NUMBER := 81;
```

```

m5 NUMBER := 74;

total NUMBER;
percentage NUMBER;
BEGIN
    total := m1 + m2 + m3 + m4 + m5;
    percentage := total / 5;

    DBMS_OUTPUT.PUT_LINE('Total Marks      : ' || total);
    DBMS_OUTPUT.PUT_LINE('Percentage    : ' || ROUND (percentage,2));

    IF percentage >= 90 THEN
        DBMS_OUTPUT.PUT_LINE('Grade : A+');
    ELSIF percentage >= 80 THEN
        DBMS_OUTPUT.PUT_LINE('Grade : A');
    ELSIF percentage >= 70 THEN
        DBMS_OUTPUT.PUT_LINE('Grade : B');
    ELSIF percentage >= 60 THEN
        DBMS_OUTPUT.PUT_LINE('Grade : C');
    ELSE
        DBMS_OUTPUT.PUT_LINE('Grade : F');
    END IF;
END;
/

```

Output

```

Total Marks      : 410
Percentage       : 82
Grade : A

```

Additional Laboratory Programs (Recommended)

For more comprehensive lab practice, you can also include:

1. Simple Interest Calculator
2. Compound Interest Calculator
3. Electricity Bill Calculation
4. Income Tax Calculator
5. Employee Bonus Calculation
6. Celsius to Fahrenheit Conversion
7. Fahrenheit to Celsius Conversion
8. Currency Conversion (INR to USD)
9. Discount Calculator
10. Net Salary Calculation (Gross Salary – Deductions)
11. BMI (Body Mass Index) Calculator
12. Profit and Loss Calculator

13. Fuel Consumption Calculator
14. EMI (Loan Installment) Calculator
15. Shopping Bill with GST and Discount

These exercises help students apply **arithmetic, relational, logical, assignment, and precedence operators** in realistic business and engineering scenarios, preparing them for both university examinations and practical database programming.

Module 4: Decision Making

Experiment 1: Even or Odd Number

Aim

To determine whether a number is even or odd using the IF-ELSE statement.

Program

```
SET SERVEROUTPUT ON;

DECLARE
    num NUMBER := 25;
BEGIN
    IF MOD(num,2)=0 THEN
        DBMS_OUTPUT.PUT_LINE(num || ' is an Even Number');
    ELSE
        DBMS_OUTPUT.PUT_LINE(num || ' is an Odd Number');
    END IF;
END;
/
```

Output

25 is an Odd Number

Experiment 2: Positive or Negative Number

Aim

To determine whether a number is positive, negative, or zero.

Program

```
SET SERVEROUTPUT ON;

DECLARE
    num NUMBER := -15;
BEGIN
    IF num > 0 THEN
        DBMS_OUTPUT.PUT_LINE('Positive Number');
    ELSIF num < 0 THEN
        DBMS_OUTPUT.PUT_LINE('Negative Number');
    END IF;
END;
```

```
        ELSE
            DBMS_OUTPUT.PUT_LINE('Zero');
        END IF;
    END;
/
```

Output

Negative Number

Experiment 3: Largest of Two Numbers

Aim

To find the largest of two numbers.

Program

```
SET SERVEROUTPUT ON;

DECLARE
    num1 NUMBER := 45;
    num2 NUMBER := 80;
BEGIN
    IF num1 > num2 THEN
        DBMS_OUTPUT.PUT_LINE(num1 || ' is the Largest Number');
    ELSE
        DBMS_OUTPUT.PUT_LINE(num2 || ' is the Largest Number');
    END IF;
END;
/
```

Output

80 is the Largest Number

Experiment 4: Largest of Three Numbers

Aim

To find the largest among three numbers.

Program


```
SET SERVEROUTPUT ON;

DECLARE
    a NUMBER := 50;
    b NUMBER := 75;
    c NUMBER := 60;
BEGIN
    IF a>b AND a>c THEN
        DBMS_OUTPUT.PUT_LINE(a || ' is the Largest Number');
    ELSIF b>a AND b>c THEN
        DBMS_OUTPUT.PUT_LINE(b || ' is the Largest Number');
    ELSE
        DBMS_OUTPUT.PUT_LINE(c || ' is the Largest Number');
    END IF;
END;
/
```

Output

75 is the Largest Number

Experiment 5: Voting Eligibility

Aim

To check whether a person is eligible to vote.

Program

```
SET SERVEROUTPUT ON;

DECLARE
    age NUMBER := 19;
BEGIN
    IF age >= 18 THEN
        DBMS_OUTPUT.PUT_LINE('Eligible for Voting');
    ELSE
        DBMS_OUTPUT.PUT_LINE('Not Eligible for Voting');
    END IF;
END;
/
```

Output

Eligible for Voting

Experiment 6: Grade Calculator

Aim

To calculate the grade based on percentage.

Program

```
SET SERVEROUTPUT ON;

DECLARE
    percentage NUMBER := 84;
BEGIN
    IF percentage >=90 THEN
        DBMS_OUTPUT.PUT_LINE('Grade : A+');
    ELSIF percentage >=80 THEN
        DBMS_OUTPUT.PUT_LINE('Grade : A');
    ELSIF percentage >=70 THEN
        DBMS_OUTPUT.PUT_LINE('Grade : B');
    ELSIF percentage >=60 THEN
        DBMS_OUTPUT.PUT_LINE('Grade : C');
    ELSIF percentage >=50 THEN
        DBMS_OUTPUT.PUT_LINE('Grade : D');
    ELSE
        DBMS_OUTPUT.PUT_LINE('Fail');
    END IF;
END;
/
```

Output

Grade : A

Experiment 7: Income Tax Calculator

Aim

To calculate income tax based on annual salary.

Program

```
SET SERVEROUTPUT ON;

DECLARE
    annual_salary NUMBER := 900000;
    tax NUMBER;
```

```

BEGIN
    IF annual_salary <= 300000 THEN
        tax := 0;
    ELSIF annual_salary <= 700000 THEN
        tax := annual_salary * 0.05;
    ELSIF annual_salary <= 1000000 THEN
        tax := annual_salary * 0.10;
    ELSE
        tax := annual_salary * 0.20;
    END IF;

    DBMS_OUTPUT.PUT_LINE('Annual Salary : ' || annual_salary);
    DBMS_OUTPUT.PUT_LINE('Income Tax      : ' || tax);
END;
/

```

Output

```

Annual Salary : 900000
Income Tax      : 90000

```

Experiment 8: Employee Bonus Calculator

Aim

To calculate employee bonus based on years of service.

Program

```

SET SERVEROUTPUT ON;

DECLARE
    salary NUMBER := 50000;
    experience NUMBER := 6;
    bonus NUMBER;
BEGIN
    IF experience >= 10 THEN
        bonus := salary * 0.20;
    ELSIF experience >= 5 THEN
        bonus := salary * 0.10;
    ELSE
        bonus := salary * 0.05;
    END IF;

    DBMS_OUTPUT.PUT_LINE('Salary : ' || salary);
    DBMS_OUTPUT.PUT_LINE('Bonus : ' || bonus);
END;
/

```

Output

Salary : 50000
Bonus : 5000

Experiment 9: Railway Ticket Discount

Aim

To calculate railway ticket discount based on passenger category.

Program

```
SET SERVEROUTPUT ON;

DECLARE
    ticket_price NUMBER := 1500;
    age NUMBER := 65;
    discount NUMBER;
    final_amount NUMBER;
BEGIN
    IF age >= 60 THEN
        discount := ticket_price * 0.40;
    ELSIF age <= 12 THEN
        discount := ticket_price * 0.50;
    ELSE
        discount := 0;
    END IF;

    final_amount := ticket_price - discount;

    DBMS_OUTPUT.PUT_LINE('Ticket Price : ' || ticket_price);
    DBMS_OUTPUT.PUT_LINE('Discount : ' || discount);
    DBMS_OUTPUT.PUT_LINE('Amount Payable : ' || final_amount);
END;
/
```

Output

```
Ticket Price : 1500
Discount : 600
Amount Payable : 900
```

Experiment 10: ATM Withdrawal Eligibility

Aim

To determine whether a customer can withdraw the requested amount.

Program

```
SET SERVEROUTPUT ON;

DECLARE
    balance NUMBER := 20000;
    withdraw_amount NUMBER := 7000;
    minimum_balance NUMBER := 1000;
BEGIN
    IF withdraw_amount <= (balance - minimum_balance) THEN
        DBMS_OUTPUT.PUT_LINE('Transaction Successful');
        DBMS_OUTPUT.PUT_LINE('Remaining Balance : ' ||
                               (balance - withdraw_amount));
    ELSE
        DBMS_OUTPUT.PUT_LINE('Insufficient Balance');
    END IF;
END;
/
```

Output

```
Transaction Successful
Remaining Balance : 13000
```

Additional Laboratory Programs (Recommended)

To strengthen students' understanding of decision-making statements, the following additional exercises can be included:

1. Leap Year Calculator
2. Divisibility Check (by 5 and 11)
3. Simple Calculator using CASE Statement
4. Electricity Bill Calculator
5. Library Fine Calculator
6. Student Scholarship Eligibility
7. Loan Eligibility Checker
8. Insurance Premium Calculator
9. Mobile Recharge Offer Calculator
10. Hotel Room Tariff Calculator
11. Online Shopping Discount Calculator
12. BMI Category Calculator
13. Blood Donation Eligibility Checker
14. Driving License Eligibility Checker
15. Employee Promotion Eligibility Checker

These laboratory exercises provide hands-on experience with **IF**, **IF-ELSE**, **Nested IF**, **ELSIF**, and **CASE** statements while exposing students to realistic business scenarios commonly encountered in enterprise applications. They also prepare students for practical examinations, technical interviews, and real-world PL/SQL programming tasks.

Module 5: Loops

Experiment 1: Print Numbers from 1 to 100

Aim

To print numbers from 1 to 100 using a FOR LOOP.

Program

```
SET SERVEROUTPUT ON;

BEGIN
    FOR i IN 1..100 LOOP
        DBMS_OUTPUT.PUT_LINE(i);
    END LOOP;
END;
/
```

Sample Output

```
1
2
3
...
100
```

Experiment 2: Print Numbers in Reverse Order (100 to 1)

Aim

To print numbers from 100 to 1 using the REVERSE keyword.

Program

```
SET SERVEROUTPUT ON;

BEGIN
    FOR i IN REVERSE 1..100 LOOP
        DBMS_OUTPUT.PUT_LINE(i);
    END LOOP;
END;
```

/

Sample Output

```
100
99
98
...
1
```

Experiment 3: Multiplication Table

Aim

To generate the multiplication table of a given number.

Program

```
SET SERVEROUTPUT ON;

DECLARE
    num NUMBER := 7;
BEGIN
    FOR i IN 1..10 LOOP
        DBMS_OUTPUT.PUT_LINE(num || ' x ' || i || ' = ' || (num*i));
    END LOOP;
END;
/
```

Sample Output

```
7 x 1 = 7
7 x 2 = 14
...
7 x 10 = 70
```

Experiment 4: Sum of First N Natural Numbers

Aim

To calculate the sum of the first N natural numbers.

Program

```
SET SERVEROUTPUT ON;

DECLARE
    n NUMBER := 10;
    total NUMBER := 0;
BEGIN
    FOR i IN 1..n LOOP
        total := total + i;
    END LOOP;

    DBMS_OUTPUT.PUT_LINE('Sum = ' || total);
END;
/
```

Sample Output

Sum = 55

Experiment 5: Factorial of a Number

Aim

To calculate the factorial of a number.

Program

```
SET SERVEROUTPUT ON;

DECLARE
    n NUMBER := 5;
    fact NUMBER := 1;
BEGIN
    FOR i IN 1..n LOOP
        fact := fact * i;
    END LOOP;

    DBMS_OUTPUT.PUT_LINE('Factorial = ' || fact);
END;
/
```

Sample Output

Factorial = 120

Experiment 6: Fibonacci Series

Aim

To generate the Fibonacci series.

Program

```
SET SERVEROUTPUT ON;

DECLARE
    a NUMBER := 0;
    b NUMBER := 1;
    c NUMBER;
BEGIN
    DBMS_OUTPUT.PUT_LINE(a);
    DBMS_OUTPUT.PUT_LINE(b);

    FOR i IN 3..10 LOOP
        c := a + b;
        DBMS_OUTPUT.PUT_LINE(c);
        a := b;
        b := c;
    END LOOP;
END;
/
```

Sample Output

```
0
1
1
2
3
5
8
13
21
34
```

Experiment 7: Prime Number

Aim

To determine whether a number is prime.

Program

```
SET SERVEROUTPUT ON;

DECLARE
    n NUMBER := 17;
    count_factor NUMBER := 0;
BEGIN
    FOR i IN 1..n LOOP
        IF MOD(n,i)=0 THEN
            count_factor := count_factor + 1;
        END IF;
    END LOOP;

    IF count_factor = 2 THEN
        DBMS_OUTPUT.PUT_LINE(n || ' is a Prime Number');
    ELSE
        DBMS_OUTPUT.PUT_LINE(n || ' is Not a Prime Number');
    END IF;
END;
/
```

Sample Output

```
17 is a Prime Number
```

Experiment 8: Armstrong Number

Aim

To check whether a number is an Armstrong number.

Program

```
SET SERVEROUTPUT ON;

DECLARE
    num NUMBER := 153;
    temp NUMBER;
    digit NUMBER;
    sum_cube NUMBER := 0;
BEGIN
    temp := num;

    WHILE temp > 0 LOOP
        digit := MOD(temp,10);
        sum_cube := sum_cube + POWER(digit,3);
        temp := TRUNC(temp/10);
    END LOOP;
```

```

        IF sum_cube = num THEN
            DBMS_OUTPUT.PUT_LINE(num || ' is an Armstrong Number');
        ELSE
            DBMS_OUTPUT.PUT_LINE(num || ' is Not an Armstrong Number');
        END IF;
    END;
/

```

Sample Output

153 is an Armstrong Number

Experiment 9: Palindrome Number

Aim

To determine whether a number is a palindrome.

Program

```

SET SERVEROUTPUT ON;

DECLARE
    num NUMBER := 121;
    temp NUMBER;
    rev NUMBER := 0;
BEGIN
    temp := num;

    WHILE temp > 0 LOOP
        rev := rev * 10 + MOD(temp,10);
        temp := TRUNC(temp/10);
    END LOOP;

    IF rev = num THEN
        DBMS_OUTPUT.PUT_LINE(num || ' is a Palindrome');
    ELSE
        DBMS_OUTPUT.PUT_LINE(num || ' is Not a Palindrome');
    END IF;
END;
/

```

Sample Output

121 is a Palindrome

Experiment 10: Reverse a Number

Aim

To reverse the digits of a number.

Program

```
SET SERVEROUTPUT ON;

DECLARE
    num NUMBER := 12345;
    rev NUMBER := 0;
BEGIN
    WHILE num > 0 LOOP
        rev := rev * 10 + MOD(num,10);
        num := TRUNC(num/10);
    END LOOP;

    DBMS_OUTPUT.PUT_LINE('Reverse Number = ' || rev);
END;
/
```

Sample Output

Reverse Number = 54321

Experiment 11: Count the Number of Digits

Aim

To count the number of digits in a number.

Program

```
SET SERVEROUTPUT ON;

DECLARE
    num NUMBER := 987654;
    count_digit NUMBER := 0;
BEGIN
    WHILE num > 0 LOOP
        count_digit := count_digit + 1;
        num := TRUNC(num/10);
    END LOOP;
```

```
        DBMS_OUTPUT.PUT_LINE('Total Digits = ' || count_digit);
END;
/
```

Sample Output

```
Total Digits = 6
```

Experiment 12: Sum of Digits

Aim

To calculate the sum of the digits of a number.

Program

```
SET SERVEROUTPUT ON;

DECLARE
    num NUMBER := 12345;
    digit NUMBER;
    total NUMBER := 0;
BEGIN
    WHILE num > 0 LOOP
        digit := MOD(num,10);
        total := total + digit;
        num := TRUNC(num/10);
    END LOOP;

    DBMS_OUTPUT.PUT_LINE('Sum of Digits = ' || total);
END;
/
```

Sample Output

```
Sum of Digits = 15
```

Experiment 13: Number Pattern Using Nested Loops

Aim

To print a number pattern using nested loops.

Program

```
SET SERVEROUTPUT ON;

BEGIN
    FOR i IN 1..5 LOOP
        FOR j IN 1..i LOOP
            DBMS_OUTPUT.PUT(j || ' ');
        END LOOP;
        DBMS_OUTPUT.NEW_LINE;
    END LOOP;
END;
/
```

Sample Output

```
1
1 2
1 2 3
1 2 3 4
1 2 3 4 5
```

Experiment 14: Star Pattern

Aim

To print a star triangle using nested loops.

Program

```
SET SERVEROUTPUT ON;

BEGIN
    FOR i IN 1..5 LOOP
        FOR j IN 1..i LOOP
            DBMS_OUTPUT.PUT('* ');
        END LOOP;
        DBMS_OUTPUT.NEW_LINE;
    END LOOP;
END;
/
```

Sample Output

```
*
* *
* * *
* * * *
```

* * * * *

Experiment 15: Floyd's Triangle

Aim

To generate Floyd's Triangle.

Program

```
SET SERVEROUTPUT ON;

DECLARE
    num NUMBER := 1;
BEGIN
    FOR i IN 1..5 LOOP
        FOR j IN 1..i LOOP
            DBMS_OUTPUT.PUT(num || ' ');
            num := num + 1;
        END LOOP;
        DBMS_OUTPUT.NEW_LINE;
    END LOOP;
END;
/
```

Sample Output

```
1
2 3
4 5 6
7 8 9 10
11 12 13 14 15
```

Additional Practice Programs

To further strengthen students' understanding of iterative control statements, you can assign the following exercises:

1. Print all even numbers from 1 to 100.
2. Print all odd numbers from 1 to 100.
3. Calculate the sum of all even numbers up to N.
4. Calculate the sum of all odd numbers up to N.
5. Find the largest digit in a given number.
6. Find the smallest digit in a given number.
7. Generate a multiplication table from 1 to 10 using nested loops.

8. Check whether a number is a Perfect Number.
9. Generate Pascal's Triangle.
10. Print hollow square and pyramid patterns.
11. Convert a decimal number to binary.
12. Generate the first N prime numbers.
13. Display the factorial series from 1! to N!.
14. Print the multiplication tables of numbers from 2 to 20.
15. Develop a menu-driven calculator using loops and CASE statements.

These exercises comprehensively cover **Simple LOOP, WHILE LOOP, FOR LOOP, Nested Loops, EXIT, EXIT WHEN**, and **CONTINUE**, providing students with a solid foundation in iterative programming using PL/SQL.